

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
ESCOLA POLITÉCNICA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO

FELIPE GROLLA FREITAS RA: 24004846
GIOVANNA LIMA SALVAGNINI RA: 25015692

RELATÓRIO DA LISTA DE EXERCÍCIOS 01

Aula 04 — Algoritmos em Linguagem C

Relatório apresentado à disciplina de Estrutura de Dados e Recuperação da Informação como requisito parcial de avaliação, contendo a resolução comentada dos dez exercícios propostos na Lista de Exercícios 01.

Disciplina: Estrutura de Dados e Recuperação da Informação

Professor(a): Everton Dias de Oliveira

Data de entrega: 02/09/2026

CAMPINAS

2026

SUMÁRIO

INTRODUÇÃO.....	4
EXERCÍCIO 1 — ORDENAÇÃO DE TRÊS NÚMEROS EM ORDEM CRESCENTE..	5
Fluxograma.....	5
Lista de funções, variáveis e comandos utilizados.....	5
Código-fonte.....	6
EXERCÍCIO 2 — LEITURA E EXIBIÇÃO DO NOME DO USUÁRIO.....	8
Fluxograma.....	8
Lista de funções, variáveis e comandos utilizados.....	8
Código-fonte.....	9
EXERCÍCIO 3 — SOMA DO TERCEIRO E DO QUINTO DÍGITO DO RA.....	11
Fluxograma.....	11
Lista de funções, variáveis e comandos utilizados.....	11
Código-fonte.....	12
EXERCÍCIO 4 — CONTAGEM REGRESSIVA A PARTIR DA SOMA DOS DÍGITOS DO RA.....	14
Fluxograma.....	14
Lista de funções, variáveis e comandos utilizados.....	14
Código-fonte.....	15
EXERCÍCIO 5 — VERIFICAÇÃO DE PARIDADE DO RA PELO ÚLTIMO DÍGITO..	17
Fluxograma.....	17
Lista de funções, variáveis e comandos utilizados.....	17
Código-fonte.....	18
EXERCÍCIO 6 — PARIDADE DA SOMA DE TRÊS NÚMEROS INTEIROS.....	20
Fluxograma.....	20
Lista de funções, variáveis e comandos utilizados.....	20
Código-fonte.....	21
EXERCÍCIO 7 — MÉDIA ARITMÉTICA DE QUATRO NÚMEROS REAIS.....	23
Fluxograma.....	23
Lista de funções, variáveis e comandos utilizados.....	23
Código-fonte.....	24
EXERCÍCIO 8 — CONVERSÃO DE TEMPERATURA CELSIUS -> FAHRENHEIT..	26
Fluxograma.....	26
Lista de funções, variáveis e comandos utilizados.....	26
Código-fonte.....	27
EXERCÍCIO 9 — IMPRESSÃO DA SEQUÊNCIA DE 1 ATÉ O NÚMERO INFORMADO.....	29
Fluxograma.....	29
Lista de funções, variáveis e comandos utilizados.....	29
Código-fonte.....	30
EXERCÍCIO 10 — CÁLCULO DO FATORIAL DE UM NÚMERO INTEIRO NÃO	

NEGATIVO.....	32
Fluxograma.....	32
Lista de funções, variáveis e comandos utilizados.....	32
Código-fonte.....	33
REFERÊNCIAS BIBLIOGRÁFICAS.....	35

INTRODUÇÃO

Este relatório documenta a resolução da Lista de Exercícios 01 da disciplina de Estrutura de Dados e Recuperação da Informação (Aula 04), composta por dez exercícios em Linguagem C que exercitam os fundamentos da construção de algoritmos: tipagem e declaração de variáveis, entrada e saída pelo console, operadores aritméticos, relacionais e lógicos, estruturas de decisão (if/else) e de repetição (for).

Cada exercício foi desenvolvido em arquivo-fonte independente, em pasta própria dentro do repositório público no [GitHub](#), comentado linha a linha conforme o enunciado e compilado com GCC sob a opção -Wall:

```
Estrutura-de-Dados/  
└─ lista1/  
    ├── ex1/ex1.c  
    ├── ex2/ex2.c  
    ├── ...  
    └─ ex10/ex10.c
```

Adotou-se ainda a validação sistemática das entradas — verificação do retorno das funções de leitura, conferência de comprimento e conteúdo, recusa de valores fora do domínio (como negativos no fatorial). Embora não exigida pelos enunciados, essa postura defensiva evita resultados silenciosamente incorretos.

O documento reserva uma seção por exercício, nesta ordem: enunciado, estratégia de solução, fluxograma, elementos da linguagem empregados e, por fim, código-fonte e telas de execução.

EXERCÍCIO 1 — ORDENAÇÃO DE TRÊS NÚMEROS EM ORDEM CRESCENTE

Enunciado:

“Elabore um programa em Linguagem C, onde dados 3 números, imprima-os em ordem crescente usando instruções if e else.”

O programa lê três números inteiros informados pelo usuário e os exibe em ordem crescente. Como o enunciado restringe a solução às instruções if e else, não é utilizado nenhum algoritmo de ordenação com vetores: em vez disso, a estrutura condicional encadeada cobre explicitamente as seis permutações possíveis entre os três valores. O operador de comparação escolhido é o menor ou igual ($\leq$), e não o menor estrito ($<$), para que entradas com números repetidos também recaiam em um dos ramos previstos.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

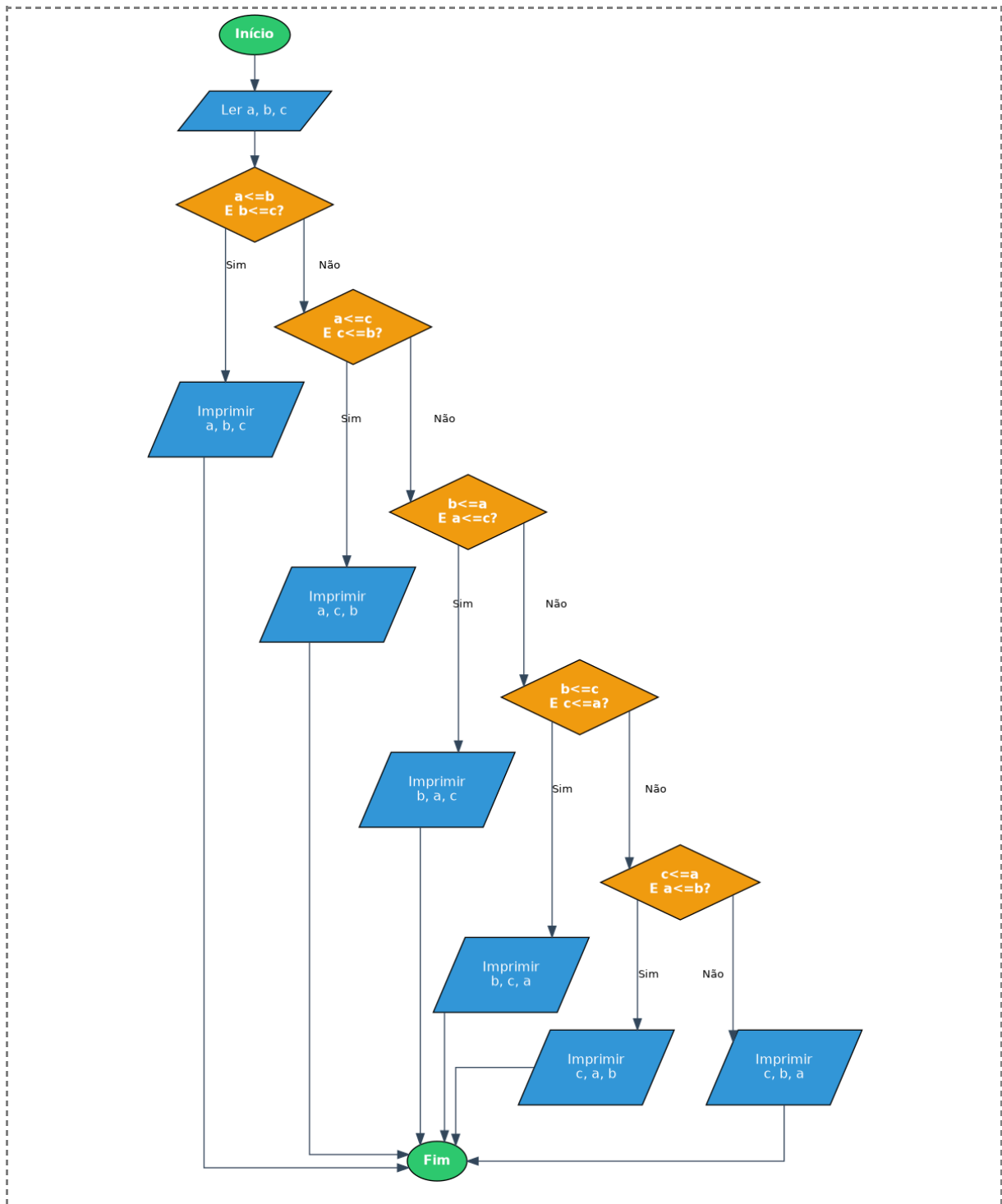


Figura 1.1 — Fluxograma do Exercício 1.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
int main(void)	Função	Função principal: ponto de partida da execução do programa.
int a, b, c	Variáveis	Armazenam os três números inteiros digitados pelo usuário.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
Máscara "%d %d %d"	Formato	Indica ao scanf que devem ser lidos três valores inteiros separados por espaço.
&a, &b, &c	Operador	Passa o endereço de memória das variáveis, para que o scanf possa gravar o valor lido dentro delas.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
else if	Comando	Testa uma nova condição somente quando as anteriores foram falsas, encadeando as seis ordens possíveis.
&&	Operador	Operador lógico E: só é verdadeiro quando as duas comparações forem verdadeiras ao mesmo tempo.
<=	Operador	Compara se um valor é menor ou igual a outro, tratando corretamente números repetidos.
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      // Definindo as variáveis para armazenar os números
6      int a, b, c;
7
8      // Solicitando ao usuário que digite três números inteiros
9      printf("Digite tres numeros inteiros: ");
10     scanf("%d %d %d", &a, &b, &c);
11
12     // Cada ramo cobre uma das 6 ordens possíveis entre a, b e
13     // Usamos <= (e não <) para que números repetidos caiam no r
14     if (a <= b && b <= c)
15     {
16         printf("Ordem crescente: %d %d %d\n", a, b, c);
17     }
18     else if (a <= c && c <= b)
19     {
20         printf("Ordem crescente: %d %d %d\n", a, c, b);
21     }
22     else if (b <= a && a <= c)
23     {
24         printf("Ordem crescente: %d %d %d\n", b, a, c);
25     }
26     else if (b <= c && c <= a)
27     {
28         printf("Ordem crescente: %d %d %d\n", b, c, a);
29     }
30     else if (c <= a && a <= b)
31     {
32         printf("Ordem crescente: %d %d %d\n", c, a, b);
33     }
34     else
35     {
36         printf("Ordem crescente: %d %d %d\n", c, b, a);
37     }
38
39     return 0;
40 }
41

```

Figura 1.2 — Código-fonte do Exercício 1.

EXERCÍCIO 2 — LEITURA E EXIBIÇÃO DO NOME DO USUÁRIO

Enunciado:

“Elabore um programa em Linguagem C, que pergunte o seu nome e imprima na tela.”

O programa solicita o nome do usuário e devolve uma saudação personalizada. A leitura é feita com `fgets`, e não com `scanf("%s")`, porque o `scanf` interrompe a leitura no primeiro espaço em branco e capturaria apenas o primeiro nome de uma entrada composta. Após a leitura, o caractere de quebra de linha gravado pelo `fgets` é substituído pelo terminador nulo, de modo que a saudação seja impressa em uma única linha.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

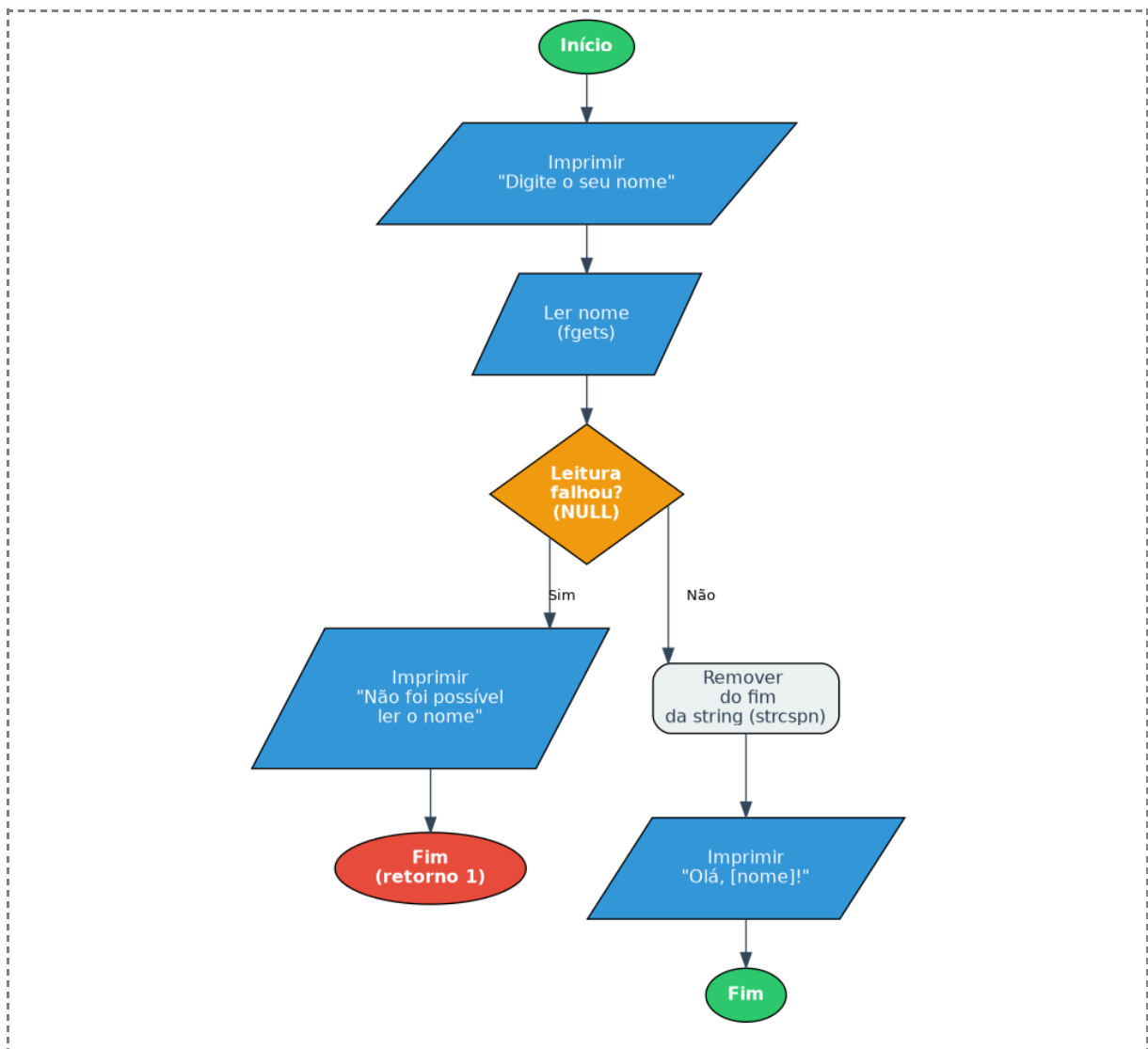


Figura 2.1 — Fluxograma do Exercício 2.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
#include <string.h>	Diretiva	Inclui a biblioteca de manipulação de cadeias de caracteres (strings).
int main(void)	Função	Função principal: ponto de partida da execução do programa.

Elemento	Categoria	O que faz no código
char nome[255]	Variável	Vetor de caracteres (string) que armazena o nome digitado, com espaço para até 254 letras.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
fgets	Função	Lê uma linha inteira digitada pelo usuário, incluindo espaços, respeitando o tamanho do vetor.
sizeof nome	Operador	Informa ao fgets o tamanho do vetor, impedindo que ele escreva além do espaço reservado.
stdin	Fluxo	Identifica a entrada padrão do programa, ou seja, o teclado.
NULL	Constante	Valor devolvido pelo fgets quando a leitura falha; permite tratar o erro.
strcspn	Função	Devolve a posição do caractere de quebra de linha dentro da string.
nome[...] = '\0'	Comando	Grava o terminador nulo no lugar do Enter, encerrando a string e removendo a quebra de linha.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main(void)
5  {
6
7      // Definindo uma variável para armazenar o nome do usuário
8      char nome[255];
9
10     // Solicitando ao usuário que digite o seu nome
11     printf("Digite o seu nome: ");
12
13     // fgets em vez de scanf("%s"): o scanf pararia no primeiro
14     // espaço e leria
15     // só "Maria" de "Maria de Souza". O sizeof limita a escrita
16     // ao vetor.
17     if (fgets(nome, sizeof nome, stdin) == NULL)
18     {
19         printf("Nao foi possivel ler o nome.\n");
20         return 1;
21     }
22
23     // O fgets guarda o Enter como '\n'. strchr devolve a posiç
24     // ão dele,
25     // e o '\0' escrito ali encerra a string, descartando a queb
26     // ra de linha.
27     nome[strchr(nome, "\n")] = '\0';
28
29     // Imprimindo uma mensagem de saudação com o nome do usuário
30     printf("Ola, %s!\n", nome);
31
32     return 0;
33 }

```

Figura 2.2 — Código-fonte do Exercício 2.

EXERCÍCIO 3 — SOMA DO TERCEIRO E DO QUINTO DÍGITO DO RA

Enunciado:

“Elabore um programa em Linguagem C, que some o terceiro e o quinto dígito do seu RA e imprima na tela a seguinte frase: “O resultado da soma é:” mostrando o resultado.”

O RA é lido como texto, e não como número inteiro, por duas razões: o formato textual permite acessar diretamente os dígitos das posições desejadas e preserva eventuais zeros à esquerda, que seriam descartados em uma variável do tipo int. Antes do cálculo, o programa valida o comprimento do RA e verifica se todos os caracteres são numéricos, evitando somas incorretas a partir de entradas malformadas.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

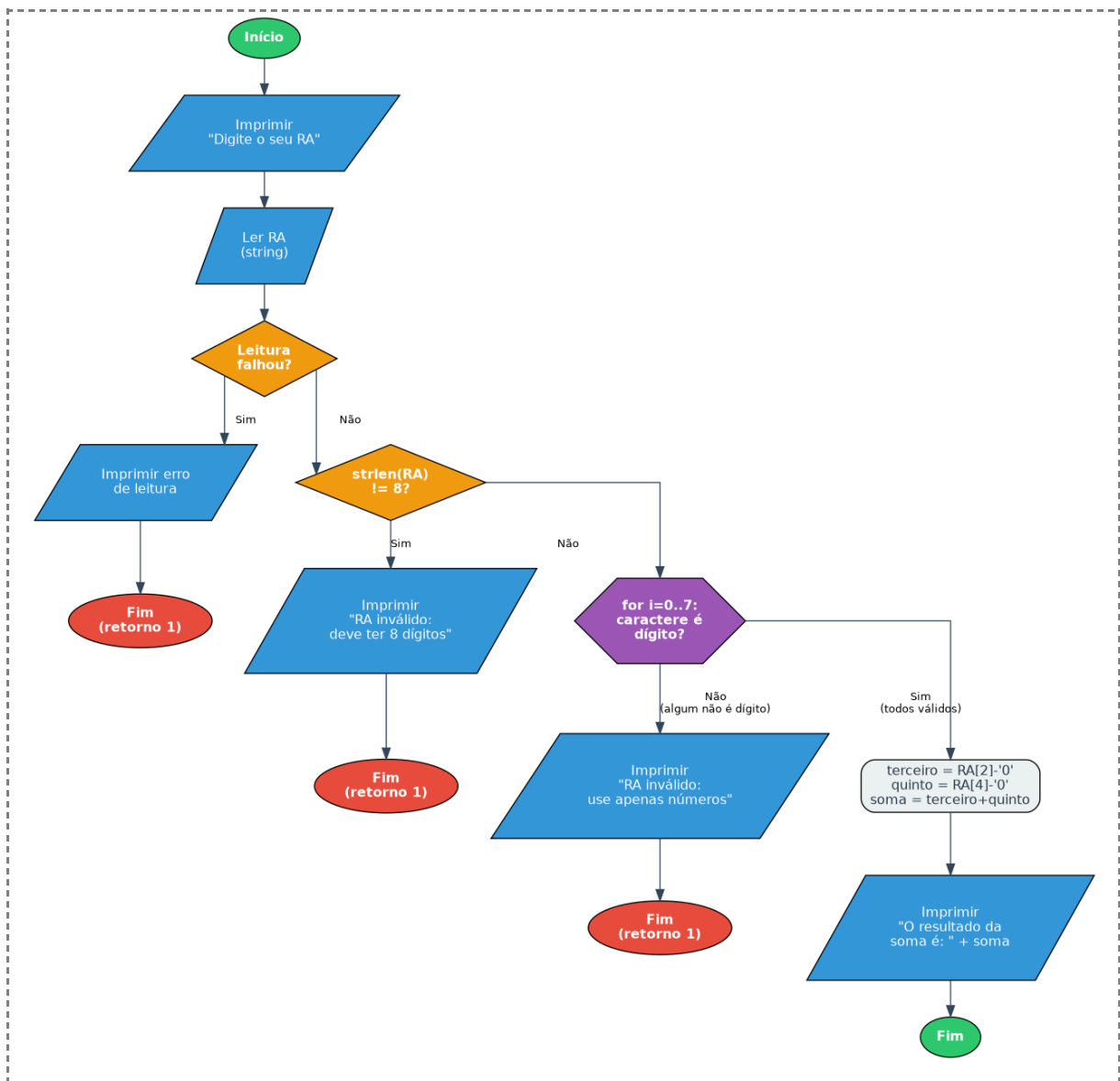


Figura 3.1 — Fluxograma do Exercício 3.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
#include <string.h>	Diretiva	Inclui a biblioteca de manipulação de cadeias de caracteres (strings).

Elemento	Categoria	O que faz no código
#include <ctype.h>	Diretiva	Inclui a biblioteca de classificação de caracteres, usada aqui pela função isdigit.
int main(void)	Função	Função principal: ponto de partida da execução do programa.
char ra[16]	Variável	Vetor de caracteres que guarda o RA como texto, permitindo acessar cada dígito pela sua posição.
int i, terceiro, quinto, soma	Variáveis	Contador do laço de validação e os dígitos extraídos com o respectivo resultado da soma.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
Máscara "%15s"	Formato	Lê no máximo 15 caracteres, evitando a escrita além do vetor reservado.
strlen	Função	Devolve a quantidade de caracteres de uma string, usada para validar o tamanho do RA.
isdigit	Função	Verifica se um caractere é um algarismo de 0 a 9.
(unsigned char)	Conversão	Converte o caractere para o tipo esperado pela função isdigit, evitando comportamento indefinido.
for	Comando	Repete um bloco de código enquanto a condição do laço for verdadeira.
ra[2] e ra[4]	Índices	Acessam o terceiro e o quinto dígito, lembrando que a contagem dos vetores começa em zero.
ra[i] - '0'	Expressão	Converte o caractere de um dígito no número que ele representa (por exemplo, '7' vira 7).
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main(void)
6 {
7     // O RA é lido como texto, não como int, porque precisamos acessar
8     // dígitos em posições específicas. Isso também preserva zeros à esquerda.
9     char ra[16];
10    int i, terceiro, quinto, soma;
11
12    // Solicitando o RA ao usuário
13    printf("Digite o seu RA (8 dígitos): ");
14
15    // O %15s limita a leitura para não escrever além do vetor.
16    // O scanf retorna quantos itens leu com sucesso.
17    if (scanf("%15s", ra) != 1)
18    {
19        printf("Nao foi possível ler o RA.\n");
20        return 1;
21    }
22
23    // Validação 1: o RA precisa ter exatamente 8 caracteres
24    if (strlen(ra) != 8)
25    {
26        printf("RA invalido: deve ter exatamente 8 digitos.\n");
27        return 1;
28    }
29
30    // Validação 2: todo caractere precisa ser numérico.
31    // Sem isso, uma entrada como "12a45678" produziria uma soma errada.
32    for (i = 0; i < 8; i++)
33    {
34        if (!isdigit((unsigned char)ra[i]))
35        {
36            printf("RA invalido: use apenas numeros.\n");
37            return 1;
38        }
39    }
40
41    // O vetor começa no índice 0, então o 3º dígito está em ra[2] e o 5º em ra[4].
42    // Subtrair '0' converte o caractere no número que ele representa ('3' - '0' == 3).
43    terceiro = ra[2] - '0';
44    quinto = ra[4] - '0';
45    soma = terceiro + quinto;
46
47    // Imprimindo o resultado da soma
48    printf("O resultado da soma e: %d\n", soma);
49
50    return 0;
51 }
52
```

Figura 3.2 — Código-fonte do Exercício 3.

EXERCÍCIO 4 — CONTAGEM REGRESSIVA A PARTIR DA SOMA DOS DÍGITOS DO RA

Enunciado:

“Elabore um programa em Linguagem C, que some o terceiro e o quinto dígito do seu RA (você pode escrever os dígitos no próprio código) e imprima na tela uma contagem regressiva começando pelo valor da soma até o 0 (zero).”

Este exercício reaproveita a extração e a validação dos dígitos do RA implementadas no exercício anterior e acrescenta uma estrutura de repetição. O laço for parte do valor da soma e decrementa a variável de controle a cada iteração; a condição de parada utiliza maior ou igual a zero ($i \geq 0$), e não maior que zero, para que o próprio zero seja impresso, conforme exige o enunciado.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

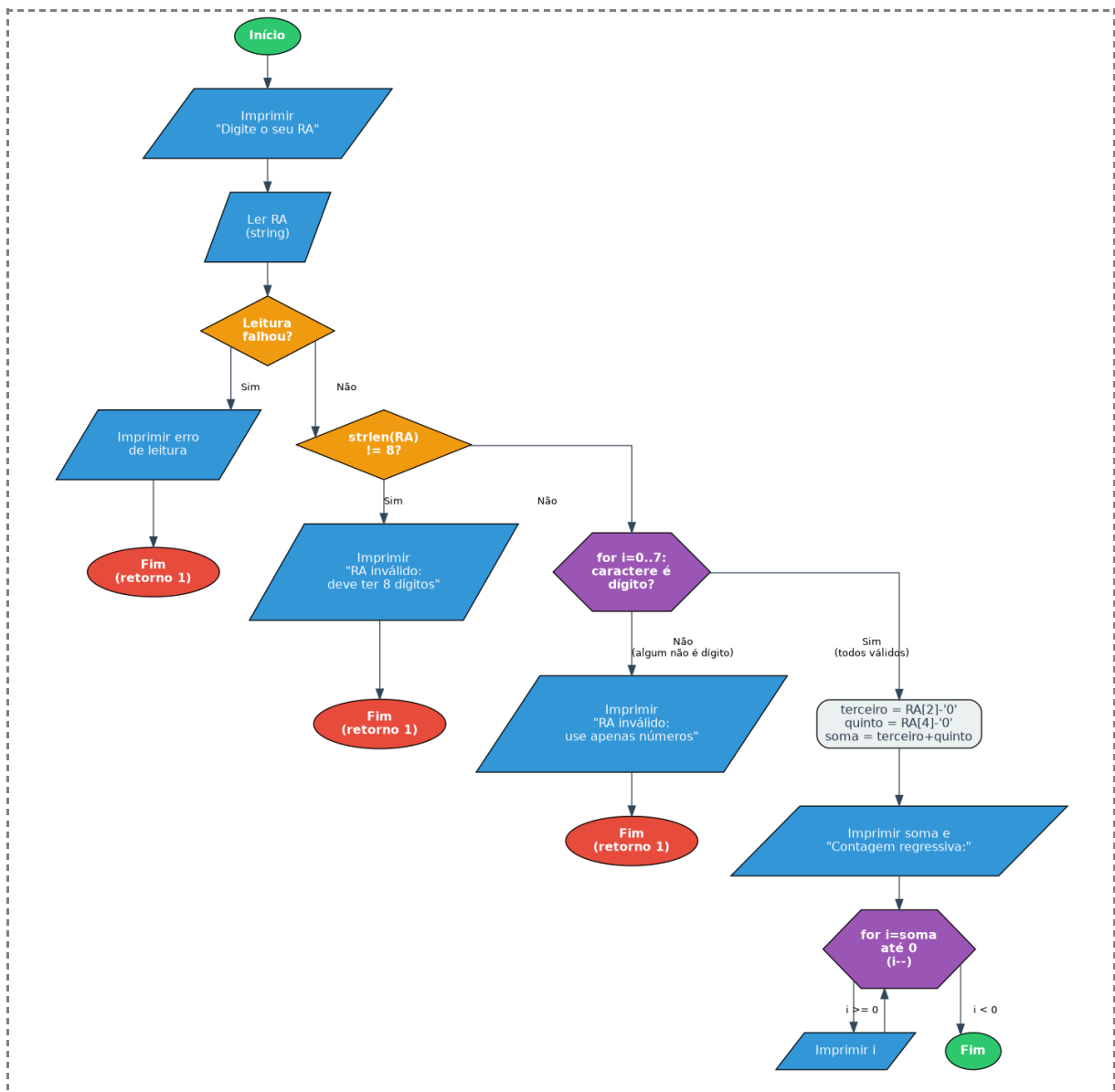


Figura 4.1 — Fluxograma do Exercício 4.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
#include <string.h>	Diretiva	Inclui a biblioteca de manipulação de cadeias de caracteres (strings).

Elemento	Categoria	O que faz no código
#include <ctype.h>	Diretiva	Inclui a biblioteca de classificação de caracteres, usada aqui pela função isdigit.
int main(void)	Função	Função principal: ponto de partida da execução do programa.
char ra[16]	Variável	Vetor de caracteres que guarda o RA como texto, permitindo acessar cada dígito pela sua posição.
int i, terceiro, quinto, soma	Variáveis	Contador do laço e os dígitos extraídos com o resultado da soma, usado como ponto de partida da contagem.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
strlen	Função	Devolve a quantidade de caracteres de uma string, usada para validar o tamanho do RA.
isdigit	Função	Verifica se um caractere é um algarismo de 0 a 9.
ra[i] - '0'	Expressão	Converte o caractere de um dígito no número que ele representa (por exemplo, '7' vira 7).
for (i = soma; i >= 0; i--)	Comando	Laço de repetição que parte do valor da soma e vai reduzindo até chegar a zero.
i--	Operador	Decrementa em uma unidade o valor do contador a cada volta do laço.
>=	Operador	Compara se o contador é maior ou igual a zero, garantindo que o zero também seja exibido.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main(void)
6 {
7     // O RA é lido como texto para permitir acessar dígitos por posição
8     char ra[16];
9     int i, terceiro, quinto, soma;
10
11     // Solicitando o RA ao usuário
12     printf("Digite o seu RA (8 dígitos): ");
13
14     // O %15s limita a leitura para não escrever além do vetor
15     if (scanf("%15s", ra) != 1)
16     {
17         printf("Não foi possível ler o RA.\n");
18         return 1;
19     }
20
21     // Validação 1: o RA precisa ter exatamente 8 caracteres
22     if (strlen(ra) != 8)
23     {
24         printf("RA inválido: deve ter exatamente 8 dígitos.\n");
25         return 1;
26     }
27
28     // Validação 2: todo caractere precisa ser numérico
29     for (i = 0; i < 8; i++)
30     {
31         if (!isdigit((unsigned char)ra[i]))
32         {
33             printf("RA inválido: use apenas números.\n");
34             return 1;
35         }
36     }
37
38     // 3º dígito em ra[2] e 5º em ra[4]; subtrair '0' converte char em número
39     terceiro = ra[2] - '0';
40     quinto = ra[4] - '0';
41     soma = terceiro + quinto;
42
43     printf("Soma do terceiro e do quinto dígito: %d\n", soma);
44     printf("Contagem regressiva:\n");
45
46     // O laço parte da soma e decrementa até 0. A condição i >= 0
47     // (e não i > 0) garante que o zero também seja impresso.
48     for (i = soma; i >= 0; i--)
49     {
50         printf("%d ", i);
51     }
52     printf("\n");
53
54     return 0;
55 }
```

Figura 4.2 — Código-fonte do Exercício 4.

EXERCÍCIO 5 — VERIFICAÇÃO DE PARIDADE DO RA PELO ÚLTIMO DÍGITO

Enunciado:

“Elabore um programa em Linguagem C, que solicite o último dígito do seu RA e imprima na tela se seu RA é par ou ímpar.”

A paridade de um número na base decimal depende exclusivamente do seu último dígito. Por essa razão, o programa não converte o RA inteiro para um valor numérico: basta isolar o caractere da última posição do vetor, convertê-lo em número e verificar o resto da sua divisão por dois. As mesmas validações de comprimento e de conteúdo numérico dos exercícios anteriores são mantidas.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

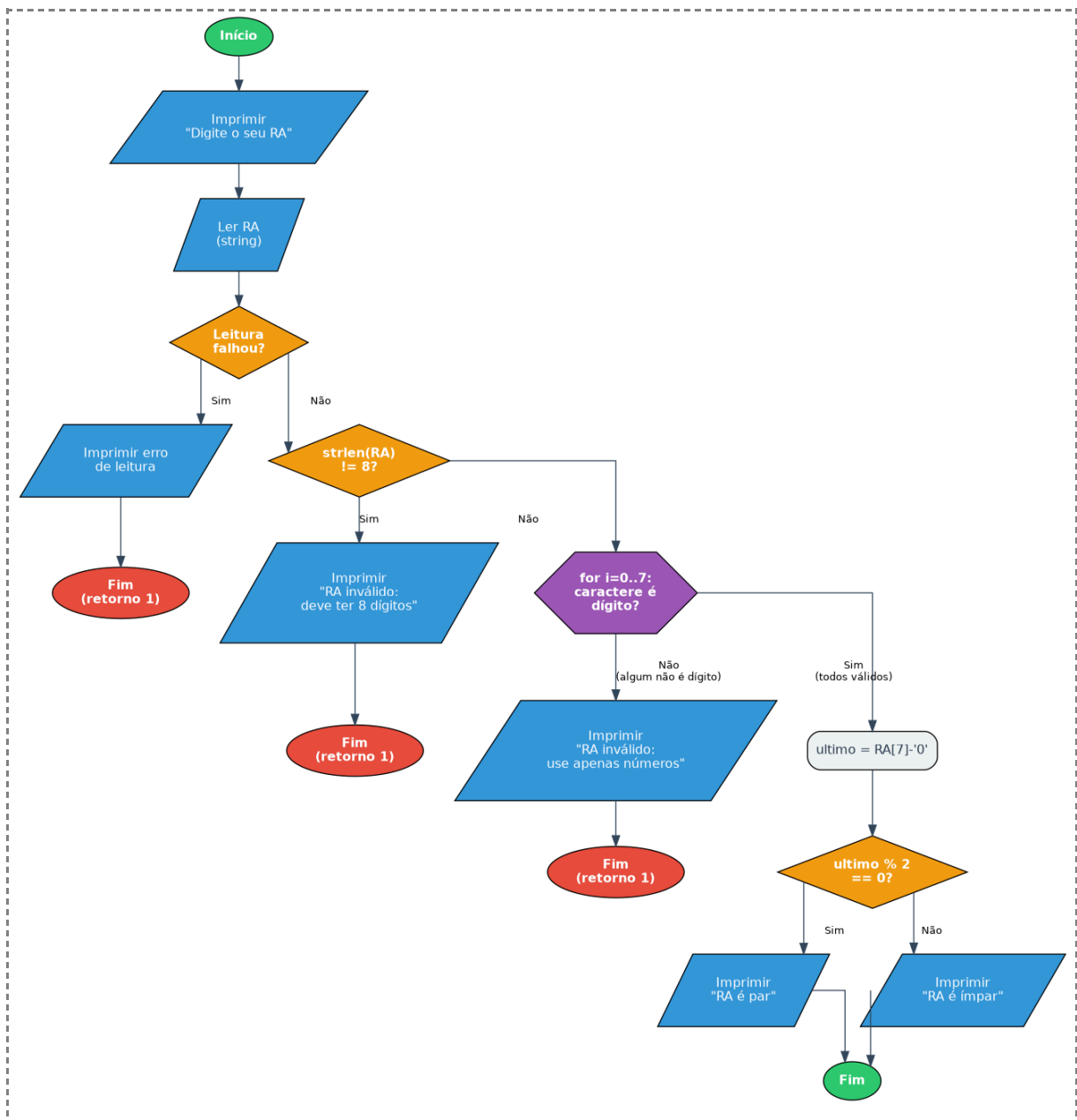


Figura 5.1 — Fluxograma do Exercício 5.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.

Elemento	Categoria	O que faz no código
#include <string.h>	Diretiva	Inclui a biblioteca de manipulação de cadeias de caracteres (strings).
#include <ctype.h>	Diretiva	Inclui a biblioteca de classificação de caracteres, usada aqui pela função isdigit.
int main(void)	Função	Função principal: ponto de partida da execução do programa.
char ra[16]	Variável	Vetor de caracteres que guarda o RA como texto, permitindo acessar cada dígito pela sua posição.
int i, ultimo	Variáveis	Contador do laço de validação e o valor numérico do último dígito do RA.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
strlen	Função	Devolve a quantidade de caracteres de uma string, usada para validar o tamanho do RA.
isdigit	Função	Verifica se um caractere é um algarismo de 0 a 9.
for	Comando	Repete um bloco de código enquanto a condição do laço for verdadeira.
ra[7]	Índice	Acessa o oitavo e último caractere do RA, do qual depende a paridade.
ra[i] - '0'	Expressão	Converte o caractere de um dígito no número que ele representa (por exemplo, '7' vira 7).
%	Operador	Devolve o resto da divisão inteira; resto zero na divisão por 2 indica número par.
ultimo % 2 == 0	Condição	Testa se o último dígito é divisível por dois, ou seja, se o RA é par.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).

Elemento	Categoria	O que faz no código
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main(void)
6 {
7     // O RA é lido como texto para permitir acessar dígitos por posição
8     char ra[16];
9     int i, ultimo;
10
11     // Solicitando o RA ao usuário
12     printf("Digite o seu RA (8 dígitos): ");
13
14     // O %15s limita a leitura para não escrever além do vetor
15     if (scanf("%15s", ra) != 1)
16     {
17         printf("Não foi possível ler o RA.\n");
18         return 1;
19     }
20
21     // Validação 1: o RA precisa ter exatamente 8 caracteres
22     if (strlen(ra) != 8)
23     {
24         printf("RA inválido: deve ter exatamente 8 dígitos.\n");
25         return 1;
26     }
27
28     // Validação 2: todo caractere precisa ser numérico
29     for (i = 0; i < 8; i++)
30     {
31         if (!isdigit((unsigned char)ra[i]))
32         {
33             printf("RA inválido: use apenas números.\n");
34             return 1;
35         }
36     }
37
38     // A paridade de um número depende apenas do último dígito,
39     // então basta olhar ra[7] em vez de converter o RA inteiro.
40     ultimo = ra[7] - '0';
41
42     // O operador % devolve o resto da divisão; resto 0 por 2 significa par
43     if (ultimo % 2 == 0)
44     {
45         printf("O último dígito é %d, então o seu RA é par.\n", ultimo);
46     }
47     else
48     {
49         printf("O último dígito é %d, então o seu RA é ímpar.\n", ultimo);
50     }
51
52     return 0;
53 }
54

```

Figura 5.2 — Código-fonte do Exercício 5.

EXERCÍCIO 6 — PARIDADE DA SOMA DE TRÊS NÚMEROS INTEIROS

Enunciado:

“Elabore um programa em Linguagem C que solicite três números inteiros ao usuário, efetue a soma dos três números e determine se esse número é par ou ímpar. O programa deve exibir a mensagem correspondente.”

O programa lê três números inteiros, calcula a soma e classifica o resultado como par ou ímpar. O teste de paridade é escrito comparando o resto da divisão com zero, e não com um, porque em Linguagem C o resto de um número negativo também é negativo: a expressão $-3 \% 2$ resulta em -1 , de modo que uma comparação com o valor 1 falharia para somas negativas. O retorno do `scanf` é verificado para garantir que os três valores foram efetivamente lidos.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

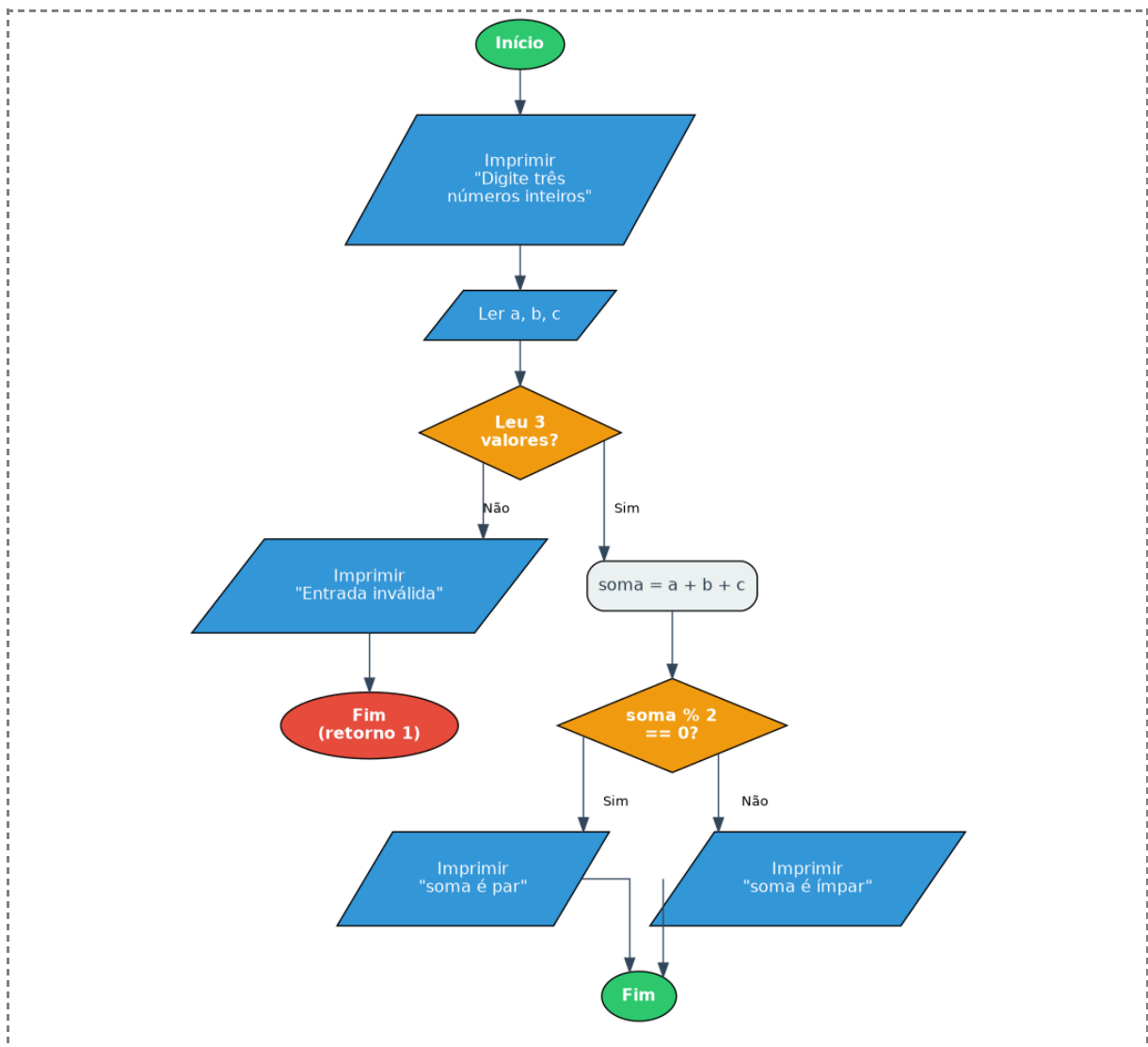


Figura 6.1 — Fluxograma do Exercício 6.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
int main(void)	Função	Função principal: ponto de partida da execução do programa.
int a, b, c, soma	Variáveis	Armazenam os três números digitados e o resultado da adição.

Elemento	Categoria	O que faz no código
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
scanf(...) != 3	Condição	Confere se os três números foram lidos com sucesso; caso contrário, a entrada é considerada inválida.
soma = a + b + c	Expressão	Calcula a soma dos três valores informados.
%	Operador	Devolve o resto da divisão inteira; resto zero na divisão por 2 indica número par.
soma % 2 == 0	Condição	Testa se a soma é divisível por dois, identificando um número par.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      // Definindo as variáveis dos três números e da soma
6      int a, b, c, soma;
7
8      // Solicitando ao usuário que digite três números inteiros
9      printf("Digite tres numeros inteiros: ");
10
11      // O scanf retorna quantos valores leu; se não forem 3, a entrada é inválida
12      if (scanf("%d %d %d", &a, &b, &c) != 3)
13      {
14          printf("Entrada invalida: digite tres numeros inteiros.\n");
15          return 1;
16      }
17
18      soma = a + b + c;
19
20      // Comparamos com 0 porque em C o resto de um número negativo é negativo:
21      // -3 % 2 vale -1, então testar "!= 1" falharia para somas negativas.
22      if (soma % 2 == 0)
23      {
24          printf("A soma e %d, que e um numero par.\n", soma);
25      }
26      else
27      {
28          printf("A soma e %d, que e um numero impar.\n", soma);
29      }
30
31      return 0;
32  }
33

```

Figura 6.2 — Código-fonte do Exercício 6.

EXERCÍCIO 7 — MÉDIA ARITMÉTICA DE QUATRO NÚMEROS REAIS

Enunciado:

“Elabore um programa em Linguagem C que peça ao usuário para inserir quatro números reais. O programa deve calcular a média desses números e exibi-la.”

Como o enunciado trata de números reais, as variáveis são declaradas com o tipo `double`, capaz de representar casas decimais. A divisão é feita pelo literal `4.0`, e não por `4`, para que a operação permaneça em ponto flutuante e não seja truncada. O resultado é exibido com duas casas decimais por meio do especificador de formato apropriado.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

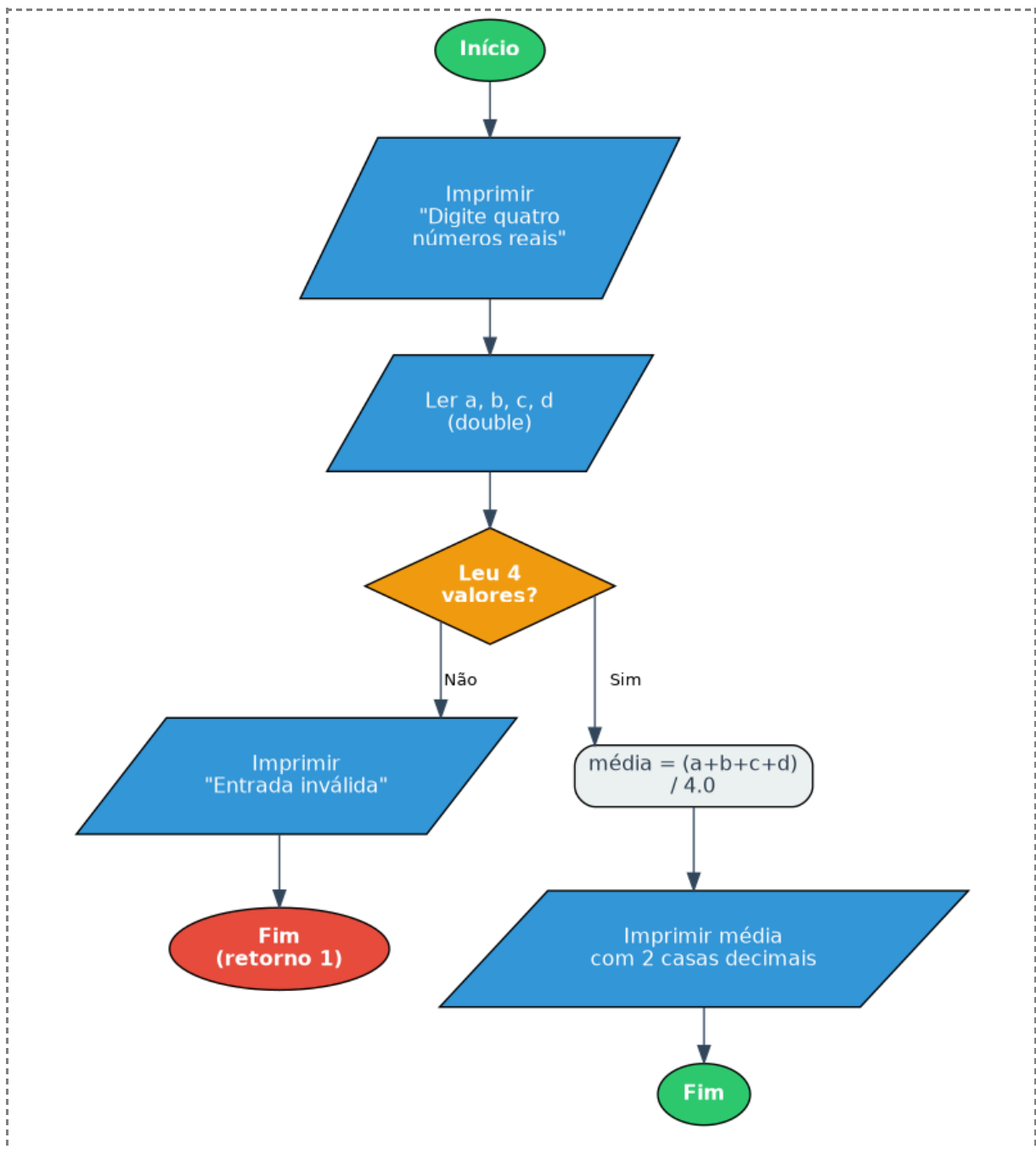


Figura 7.1 — Fluxograma do Exercício 7.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.

Elemento	Categoria	O que faz no código
int main(void)	Função	Função principal: ponto de partida da execução do programa.
double a, b, c, d, media	Variáveis	Armazenam os quatro números reais digitados e a média calculada.
double	Tipo	Tipo de dado usado para números com casas decimais, ao contrário do int, que guarda apenas inteiros.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
Máscara "%lf"	Formato	Indica ao scanf que o valor lido deve ser gravado em uma variável do tipo double.
(a + b + c + d) / 4.0	Expressão	Soma os quatro valores e divide por quatro em ponto flutuante, obtendo a média.
Máscara "%.2f"	Formato	Imprime o número real com exatamente duas casas decimais.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      // double armazena números reais (com casas decimais), diferente de int
6      double a, b, c, d, media;
7
8      // Solicitando ao usuário que digite quatro números reais
9      printf("Digite quatro numeros reais: ");
10
11     // %lf é o formato de leitura para double (%f seria para float)
12     if (scanf("%lf %lf %lf %lf", &a, &b, &c, &d) != 4)
13     {
14         printf("Entrada invalida: digite quatro numeros reais.\n");
15         return 1;
16     }
17
18     // Dividimos por 4.0 (e não por 4) para manter a divisão em ponto flutuante
19     media = (a + b + c + d) / 4.0;
20
21     // %.2f imprime o resultado com duas casas decimais
22     printf("A média dos quatro numeros e: %.2f\n", media);
23
24     return 0;
25 }
26

```

Figura 7.2 — Código-fonte do Exercício 7.

EXERCÍCIO 8 — CONVERSÃO DE TEMPERATURA CELSIUS -> FAHRENHEIT

Enunciado:

*“Elabore um programa em Linguagem C, que solicite ao utilizador a temperatura em graus Celsius e a converta para Fahrenheit. A fórmula de conversão é: $F = (C * 9/5) + 32$.”*

O programa aplica diretamente a fórmula de conversão fornecida no enunciado. O detalhe relevante da implementação está na escrita da fração: os literais são declarados como 9.0 e 5.0 para forçar a divisão em ponto flutuante, pois a divisão entre os inteiros 9 e 5 resultaria em 1 e produziria uma conversão incorreta. O valor convertido é apresentado com duas casas decimais.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

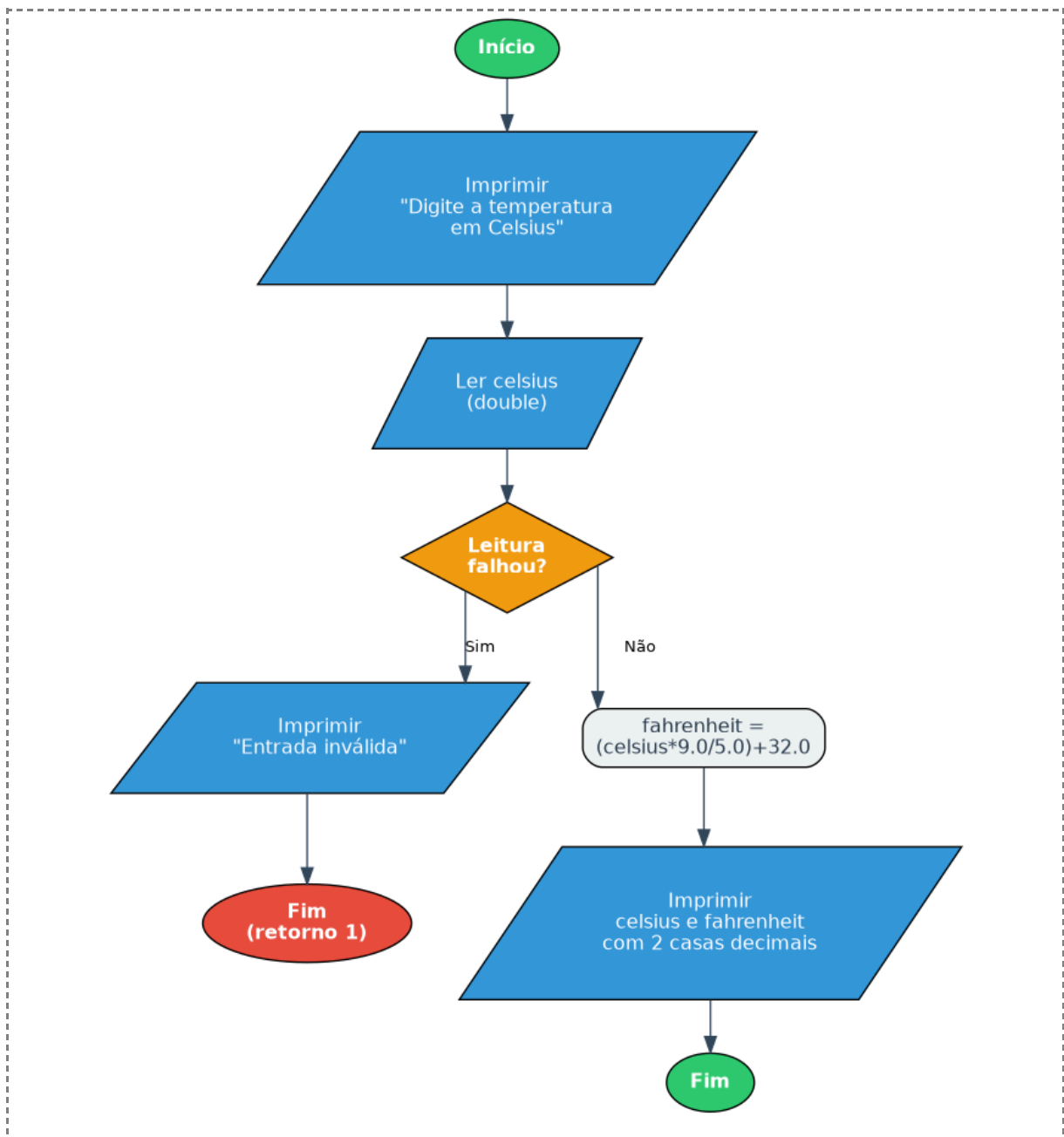


Figura 8.1 — Fluxograma do Exercício 8.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.

Elemento	Categoria	O que faz no código
int main(void)	Função	Função principal: ponto de partida da execução do programa.
double celsius, fahrenheit	Variáveis	Armazenam a temperatura informada em Celsius e o resultado da conversão em Fahrenheit.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
Máscara "%lf"	Formato	Indica ao scanf que o valor lido deve ser gravado em uma variável do tipo double.
(celsius * 9.0 / 5.0) + 32.0	Expressão	Aplica a fórmula de conversão, usando literais reais para evitar a divisão inteira.
Máscara "%.2f"	Formato	Imprime a temperatura com duas casas decimais.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     // double para aceitar temperaturas com casas decimais
6     double celsius, fahrenheit;
7
8     // Solicitando a temperatura em Celsius
9     printf("Digite a temperatura em graus Celsius: ");
10
11     // %lf é o formato de leitura para double
12     if (scanf("%lf", &celsius) != 1)
13     {
14         printf("Entrada invalida: digite um numero.\n");
15         return 1;
16     }
17
18     // Fórmula: F = (C * 9/5) + 32.
19
20     // Escrevemos 9.0 / 5.0 para forçar divisão real; 9 / 5 entre inteiros daria 1.
21     fahrenheit = (celsius * 9.0 / 5.0) + 32.0;
22
23     // Imprimindo o resultado com duas casas decimais
24     printf("%.2f C equivale a %.2f F\n", celsius, fahrenheit);
25
26     return 0;
27 }
```

Figura 8.2 — Código-fonte do Exercício 8.

EXERCÍCIO 9 — IMPRESSÃO DA SEQUÊNCIA DE 1 ATÉ O NÚMERO INFORMADO

Enunciado:

“Elabore um programa em Linguagem C, que peça ao utilizador para inserir um número inteiro positivo, maior do que 1 (um). O programa deve, então, imprimir todos os números inteiros de 1 até ao número inserido pelo utilizador.”

O programa valida a restrição imposta pelo enunciado, recusando valores menores ou iguais a um, e em seguida percorre a sequência com um laço de repetição. A condição de parada é escrita com menor ou igual ($i \leq n$), e não com menor estrito, para que o próprio número informado pelo usuário seja o último valor impresso.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

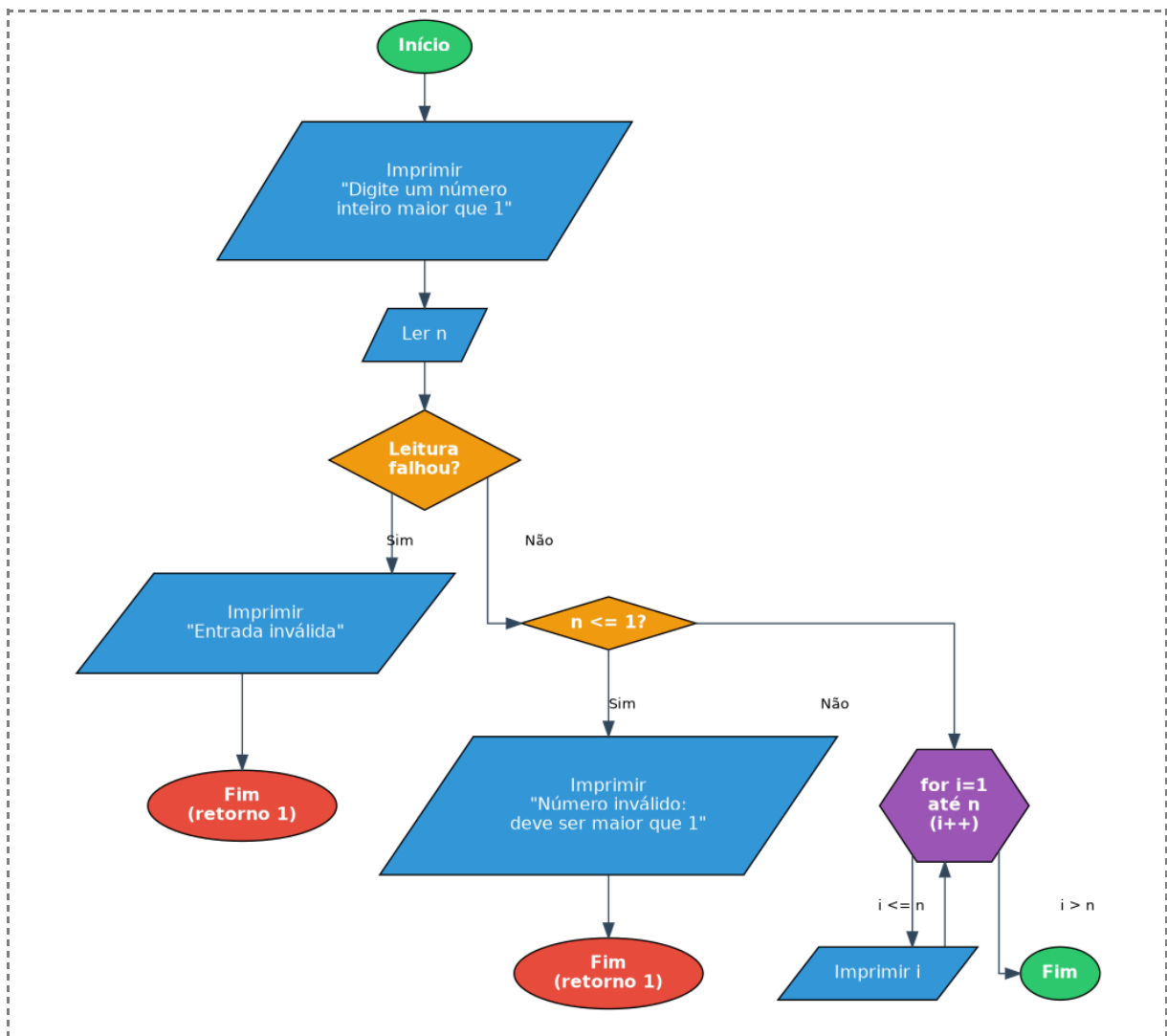


Figura 9.1 — Fluxograma do Exercício 9.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
int main(void)	Função	Função principal: ponto de partida da execução do programa.
int n, i	Variáveis	Guardam, respectivamente, o limite informado pelo usuário e o contador do laço.

Elemento	Categoria	O que faz no código
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
scanf(...) != 1	Condição	Confere se um número inteiro foi realmente lido antes de prosseguir.
n <= 1	Condição	Rejeita valores que não atendem à exigência do enunciado de ser maior que um.
for (i = 1; i <= n; i++)	Comando	Laço que percorre todos os inteiros de 1 até o número informado.
i++	Operador	Incrementa o contador em uma unidade a cada volta do laço.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      // n guarda o limite digitado e i é o contador do laço
6      int n, i;
7
8      // Solicitando um número inteiro maior que 1
9      printf("Digite um numero inteiro positivo maior que 1: ");
10
11     // O scanf retorna 1 se conseguiu ler um inteiro
12     if (scanf("%d", &n) != 1)
13     {
14         printf("Entrada invalida: digite um numero inteiro.\n");
15         return 1;
16     }
17
18     // O enunciado exige um número maior que 1, então recusamos o resto.
19     // O return 1 encerra o programa sinalizando erro.
20     if (n <= 1)
21     {
22         printf("Numero invalido: ele precisa ser maior que 1.\n");
23         return 1;
24     }
25
26     // O laço vai de 1 até n. A condição i <= n (e não i < n)
27     // garante que o próprio n seja impresso.
28     for (i = 1; i <= n; i++)
29     {
30         printf("%d ", i);
31     }
32     printf("\n");
33
34     return 0;
35 }
36

```

Figura 9.2 — Código-fonte do Exercício 9.

EXERCÍCIO 10 — CÁLCULO DO FATORIAL DE UM NÚMERO INTEIRO NÃO NEGATIVO

Enunciado:

“Elabore um programa em Linguagem C, que peça ao usuário para inserir um número inteiro não negativo e calcule o fatorial desse número.”

O fatorial é calculado de forma iterativa, por meio de um laço de repetição que acumula o produto dos inteiros de 2 até o número informado. A variável acumuladora é declarada como unsigned long long, de 64 bits, porque o fatorial cresce muito rapidamente; ainda assim, o programa recusa valores acima de 20, pois 21! ultrapassa a capacidade desse tipo e provocaria um estouro silencioso com resultado incorreto. O acumulador é inicializado em 1, elemento neutro da multiplicação, o que faz com que o caso 0! seja corretamente tratado como 1 sem nenhuma verificação adicional.

Fluxograma

O diagrama de blocos a seguir representa a lógica de execução do algoritmo, do início ao fim do programa.

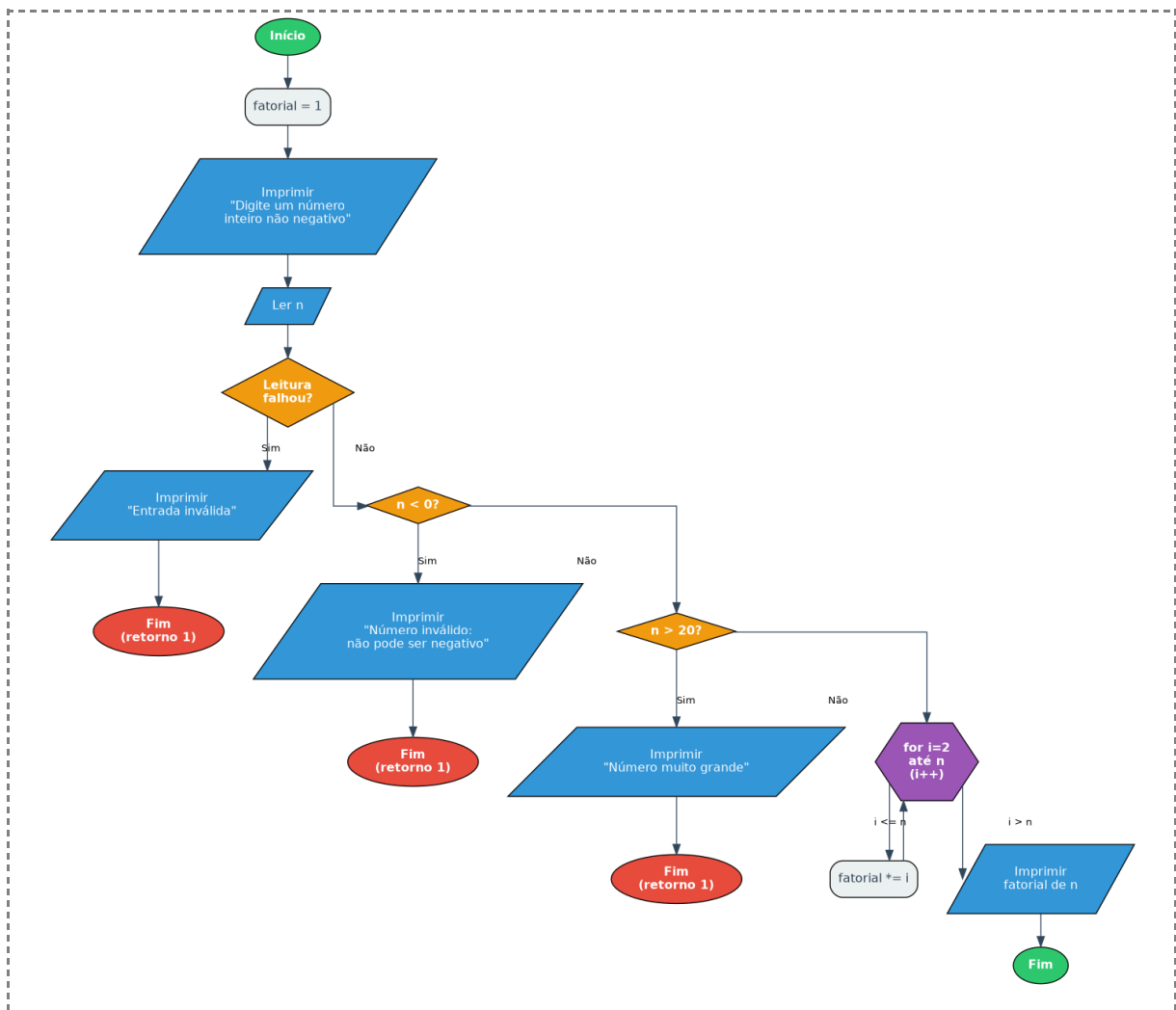


Figura 10.1 — Fluxograma do Exercício 10.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
int main(void)	Função	Função principal: ponto de partida da execução do programa.
int n, i	Variáveis	Guardam o número informado pelo usuário e o contador do laço.

Elemento	Categoria	O que faz no código
unsigned long long fatorial	Variável	Acumula o resultado do fatorial; usa 64 bits sem sinal para comportar valores muito grandes.
1ULL	Literal	Valor inicial do acumulador, escrito no tipo unsigned long long; é o elemento neutro da multiplicação.
printf	Função	Mostra uma mensagem (ou o valor de uma variável) na tela.
scanf	Função	Lê o dado digitado pelo usuário no teclado e o guarda em uma variável.
n < 0	Condição	Rejeita números negativos, para os quais o fatorial não é definido.
n > 20	Condição	Rejeita números cujo fatorial não caberia na variável, evitando resultado incorreto por estouro.
for (i = 2; i <= n; i++)	Comando	Laço que multiplica sucessivamente os inteiros de 2 até n.
fatorial *= i	Expressão	Multiplica o acumulador pelo valor atual do contador e guarda o resultado nele mesmo.
Máscara "%llu"	Formato	Imprime na tela um valor do tipo unsigned long long.
if / else	Comando	Executa um bloco de código apenas quando a condição testada for verdadeira; caso contrário, executa o bloco alternativo.
return 1	Comando	Encerra o programa antecipadamente sinalizando erro (entrada inválida).
return 0	Comando	Encerra o programa informando ao sistema que tudo ocorreu bem.

Código-fonte

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int n, i;
6
7
8      // unsigned long long (64 bits) é usado porque o fatorial cresce muito rápido.
9      // Começa em 1 porque é o elemento neutro da multiplicação.
10     unsigned long long fatorial = 1ULL;
11
12     // Solicitando um número inteiro não negativo
13     printf("Digite um numero inteiro nao negativo: ");
14
15     if (scanf("%d", &n) != 1)
16     {
17         printf("Entrada invalida: digite um numero inteiro.\n");
18         return 1;
19     }
20
21     // Fatorial não é definido para números negativos
22     if (n < 0)
23     {
24         printf("Numero invalido: ele nao pode ser negativo.\n");
25         return 1;
26     }
27
28     // 21! ultrapassa o limite do unsigned long long e o resultado daria a volta,
29     // imprimindo um valor errado. Por isso barramos antes de calcular.
30     if (n > 20)
31     {
32         printf("Numero muito grande: o fatorial so cabe na memoria ate 20.\n");
33         return 1;
34     }
35
36     // O laço começa em 2 porque multiplicar por 1 não muda nada.
37
38     // Quando n é 0 ou 1 ele nem executa, e fatorial continua valendo 1 (0! = 1).
39     for (i = 2; i <= n; i++)
40     {
41         fatorial *= (unsigned long long)i;
42     }
43
44     // %llu é o formato de impressão para unsigned long long
45     printf("O fatorial de %d e: %llu\n", n, fatorial);
46
47     return 0;
48 }

```

Figura 10.2 — Código-fonte do Exercício 10.

REFERÊNCIAS BIBLIOGRÁFICAS

OLIVEIRA, Everton Dias. Aula 04 — Lista de Exercícios 01. Campinas: Pontifícia Universidade Católica de Campinas, 2026. Material didático da disciplina Estrutura de Dados e Recuperação da Informação.

GROLLA, Felipe. Estrutura-de-Dados: repositório da disciplina. GitHub, 2026. Disponível em: <https://github.com/Grolla05/Estrutura-de-Dados>. Acesso em: 30 ago. 2026